CALM Lab useRguide

Jason Tsukahara

2026-10-18

Table of contents

Welcome 4

What is R? 5

Install and Setup 6

Install Software . . . 6
Install R . . . 6
Install Positron . . . 6
Update Software . . . 7
Update R . . . 7
Update Positron . . . 7
Getting to Know Positron . . . 7
Configure Positron . . . 7
Install Extensions . . . 8
Opening Settings . . . 8
Files & whitespace . . . 8
Editor . . . 8
Quarto . . . 9
Positron & keybindings . . . 9
Workbench, window & appearance . . . 9
Git & GitHub . . . 10
Special cases . . . 10

I Intro to R 11

1 R Basics 12

1.1 Prepare . . . 12
1.2 New File . . . 12
1.3 Creating R objects . . . 13
1.4 Classes . . . 15
1.5 Vectors . . . 16
1.6 Factors . . . 18
1.7 Lists . . . 19
1.8 Data Frames . . . 21

1.9 If...then Statements 23
1.10 More Resources 25

References **26**

Welcome

This is the useRguide for the CALM Lab. It is intended to be a resource for new lab members to get up to speed with the tools and workflows that we use in the lab. It is also a reference for existing lab members to refresh their knowledge or learn new skills.

What is R?

According to the R website

> R is a free software environment for statistical computing and graphics.

Let's unpack this.

- Free is nice!
- Statistical computing is a general term referring to a large class of techniques that involves statistics, machine learning, and more.
- Graphics refers to creating visualizations of data and output from statistical computing.

It's really all about **data**. We live in a world where data is everywhere. Our personal lives, the media, and our work are all filled with data. That data is hidden behind numbers stored in spreadsheets, databases, and other formats. A fundamental skill in the modern world is the ability to analyze and visualize data. R is a tool that can help you do just that.

R is a programming language. There are pros and cons to using a programming language over Excel and GUI (Graphical User Interface) applications with click-and-drag functionality. The pros are related to the fact that programming languages are more flexible, powerful, and reproducible than GUI applications. The cons are that programming languages have a steeper learning curve than GUI applications and there are often more than one way to do the same thing in a programming language (possibly leading to confusion).

Debates on which programming language you should learn and use are rampant on the internet and social media. Regardless of which programming language you learn, working with data through code provides a challenge that will open you up to the world of **data**. You will gain an appreciation for the level of mundanity and complexity that is involved in processing the vast amounts of data in this world. Through working with data yourself, you will also become a better consumer of data and information that you see in your own work, the news, and social media.

It all starts with gaining a basic level of proficiency in 1) working with data, 2) data visualization, and 3) statistical computation. The first few chapters of this guide will focus on these three areas. After that, you will be able to explore more advanced levels of data analysis and visualization.

Install and Setup

First things first, you need to install R and Positron and configure your environment in a way that is conducive to our workflow in the lab.

If you do not have R or Positron installed on your computer, then follow the install instructions below.

If you already have R or Positron installed on your computer, then you may want to update your installations.

Install Software

Install R

Download and install R from the R Project website.

1. Go to the R Project website.
2. Click on the "CRAN" link under "Download, Packages" in the left-hand menu.
3. Choose the 0-Cloud mirror option.
4. Select the **base** installation (for Windows) or the **Latest Release** (for Mac) option.
5. Open and run the installer file that you downloaded. Follow the prompts to complete the installation.

Install Positron

Download and install Positron from the Positron website.

1. Go to the Positron website.
2. Click on the "Download" button.
3. Select the appropriate version for your operating system.
4. Open and run the installer file that you downloaded. Follow the prompts to complete the installation.

Update Software

Update R

Warning: When updating R (not RStudio), it may remove all packages you have installed.

First check what version of R you have installed by opening Positron and checking the R version number in the console when you start it up. If you have an R version older than 4.6.0, then you should update your R installation.

Use the installr package to update R on Windows. To do this, run the following code in the Positron console:

```
install.packages("installr")
library(installr)
updateR()
```

Update Positron

To update Positron, go to the Positron website and download the latest version for your operating system. Then, open and run the installer file that you downloaded. Follow the prompts to complete the installation.

Getting to Know Positron

While R is the programming language that we will be using, Positron is the software that we will use to write and run R code. Positron is an Integrated Development Environment (IDE) for R. It provides a user-friendly interface for writing and running R code, as well as tools for debugging and visualizing data.

Open Positron and go through the Welcome Tour and Setup Guides to get familiar with the interface and features.

Configure Positron

Have a lab member or Dr. Tsukahara walk you through the Positron interface, and help you configure the settings to match the lab's workflow. The following sections provide a summary of the recommended settings.

Install Extensions

To install an extension, click the **Extensions** icon (left sidebar) and search for the extension name.

Feel free to explore the available extensions, but the following are recommended for our lab workflow:

- **Fira Code** — a fancy monospaced font with ligatures (e.g., `!=` becomes).
- **Material Icon Theme** — icon themes change the look of folders and files.
- **One Dark Pro** — a dark theme for your IDE.
- **GitHub Theme** — a light theme for your IDE.
- **vscode-pets** — adds a cute pet to your IDE.

Opening Settings

- Press Cmd, or click the gear icon (bottom-left) → **Settings**.
- In the Settings window, click the **Search settings** bar at the top and type in the name of the setting you want to change. The list of settings will filter as you type.

Files & whitespace

Setting	Search for	Action
Files: Associations	`files associations`	Click **Add Item** → Item: `renv.lock`, Value: `json`
Files: Trim Trailing Whitespace	`trim trailing`	Check
Files: Insert Final Newline	`insert final newline`	Check
Files: Trim Final Newlines	`trim final newlines`	Check

Editor

Setting	Search for	Action
Editor: Font Family	`font family`	Set to `Fira Code` (keep existing fallbacks after it)
Editor: Font Ligatures	`font ligatures`	Check
Editor: Font Size	`editor font size`	Set to `14`
Editor: Format On Type	`format on type`	Check
Editor: Tab Size	`tab size`	Set to `2`

Setting	Search for	Action
Editor: Detect Indentation	`detect indentation`	**Uncheck**
Editor: Auto Indent	`auto indent`	Dropdown → `full`
Editor: Wrapping Indent	`wrapping indent`	Dropdown → `indent`
Editor: Default Formatter	`default formatter`	Dropdown → **Positron R** (`positron.positron-r`)

Quarto

Setting	Search for	Action
Quarto › Cells › Background: Color	`quarto background color`	Dropdown → `useTheme`
Quarto › Visual Editor: Markdown Wrap	`quarto markdown wrap`	Dropdown → `sentence`
Quarto › Visual Editor: Default List Spacing	`quarto list spacing`	Dropdown → `tight`

Positron & keybindings

Setting	Search for	Action
Positron › Assistant: Enable	`positron assistant enable`	Check
Workbench › Keybindings: Rstudio Keybindings	`rstudio keybindings`	Check

Workbench, window & appearance

Setting	Search for	Action
Workbench: Startup Editor	`startup editor`	Dropdown → `readme`
Symbols: Hides Explorer Arrows	`hides explorer arrows`	**Uncheck** (needs Symbols extension)
Window: Restore Windows	`restore windows`	Dropdown → `none`
Window: Close When Empty	`close when empty`	Check

Icon theme (`vscode-icons`): set this via the picker rather than search — open the Command Palette (**Cmd/Ctrl+Shift+P**) → **Preferences: File Icon Theme** → choose **VSCode Icons**.

Git & GitHub

Setting	Search for	Action
Git: Autofetch	`git autofetch`	Check
Git: Enable Smart Commit	`smart commit`	Check
Git: Confirm Sync	`confirm sync`	**Uncheck**
GitHub › Copilot › Next Edit Suggestions: Enabled	`next edit suggestions`	Check (needs Copilot extension)

Special cases

Markdown-only whitespace override (`"[markdown]": { trimTrailingWhitespace: false }`): in the Settings search bar type `@lang:markdown`, find **Trim Trailing Whitespace**, and **uncheck** it. The funnel scopes that single change to Markdown only and writes the language block for you.

Editor: Rulers (`[80]`): this array has no graphical control. Searching for `rulers` only offers an **Edit in settings.json** link, so this one entry must stay in JSON — add `"editor.rulers": [80]`.

Part I

Intro to R

1 R Basics

Intro to R

The main objective of this section is to get you familiar with the basics of R. This includes basic syntax, creating objects, understanding the different classes of objects, and understanding the different data structures in R.

1.1 Prepare

Before starting this section:

Read the What is R? page

Read the Install and Setup page

1.2 New File

File -> New File -> R File

This should have opened a blank **Script** window called **Untitled**.

The **Script** window is a file where you are saving your code. This is where you will write, edit, delete, and re-write your code.

To follow along with the tutorial, you should type (for now, resist just copying and pasting) the lines of code I display in the tutorial into your script.

Save your empty script somewhere on your computer

1.3 Creating R objects

In R, everything that exists is an object and everything you do to objects are functions. You can define an object using the assignment operator `<-`.

Everything on the left hand side of the `<-` assignment operator is an object. Everything on the right hand side of `<-` are functions or values. Go ahead and type the following two lines of code in your script

```
string <- "hello"
string
```

```
[1] "hello"
```

You can execute/run a line of code by placing the cursor anywhere on the line and press `Ctrl + Enter`. Go ahead an run the two lines of code.

In this example, the first line creates a new object called `string` with a value of "hello". The second line simply prints the output of `string` to the **Console window**. In the second line there is no assignment operator. When there is no `<-` this means you are essentially just printing to the console. You can't do anything with stuff that is just printed to the console, it is just for viewing purposes.

For instance, if I wanted to calculate `1 + 2` I could do this by printing it to the console

```
1 + 2
```

```
[1] 3
```

However, if I wanted to do something else with the result of that calculation then I would not be able to unless I assigned the result to an object using `<-`

```
result <- 1 + 2
result <- result * 5

result
```

```
[1] 15
```

The point is, you are almost always going to assign the result of some function or value to an object. Printing to the console is not very useful. Almost every line of code, then, will have an object name on the left hand side of `<-` and a function or value on the right hand side of `<-`

In the first example above, notice how I included `" "` around hello. This tells R that hello is a string, not an object. If I were to not include `" "`, then R would think I am calling an object. And since there is no object with the name hello it will print an error

```
string <- hello
```

```
Error:
! object 'hello' not found
```

Do not use `" "` for Numerical values

```
a <- "5" + "1"
```

```
Error in `"5" + "1"`:
! non-numeric argument to binary operator
```

You can execute lines of code by:

1. Typing them directly into the Console window
2. Typing them into the Script window and then on that line of code pressing **Ctrl+Enter**. With **Ctrl+Enter** you can execute one line of your code at a time.
3. Clicking on **Source** at the top right of the Script window. This will run ALL the lines of code contained in the script file.

It is important to know that EVERYTHING in R is **case sensitive**.

```
a <- 5

a + 5
```

```
[1] 10
```

```
A + 5
```

```
Error:
! object 'A' not found
```

1.4 Classes

Classes are types of values that exist in R:

- character `"hello"`, `"19"`
- numeric (or double) `2`, `32.55`
- integer `5`, `99`
- logical `TRUE`, `FALSE`

To evaluate the class of an object you can use the `typeof()`

```
typeof(a)
```

```
[1] "double"
```

To change the class of values in an object you can use the function `as.character()` , `as.numeric()` , `as.double()` , `as.integer()` , `as.logical()` functions.

```
as.integer(a)
```

```
[1] 5
```

```
as.character(a)
```

```
[1] "5"
```

```
as.numeric("hello")
```

```
[1] NA
```

1.5 Vectors

Okay so now I want to talk about creating more interesting objects than just `a <- 5`. If you are going to do anything in R it is important that you understand the different data types and data structures you can use in R. I will not cover all of them in this tutorial. For more information on data types and structures see this nice Introduction to R

Vectors contain elements of data. The length of a vector is the number of elements in the vector. For instance, the variable `a` we created earlier is actually a vector of length 1. It contains one element with a value of 5. Now let's create a vector with more than one element.

```
b <- c(1,3,5)
```

`c()` is a function. Functions contain arguments that are inputs for the function. Arguments are separated by commas. In this example the `c()` function concatenates the arguments (`1, 3, 5`) into a vector. We are passing the result of this function to the object b. What do you think the output of b will look like?

```
b
```

```
[1] 1 3 5
```

You can see that we now have a vector that contains 3 elements; `1, 3, 5`. If you want to reference the value of specific elements of a vector you use brackets `[ ]`. For instance,

```
b[2]
```

```
[1] 3
```

The value of the second element in vector `b` is 3. Let's say we want to grab only the 2nd and 3rd elements. We can do this at least two different ways.

```
b[2:3]
```

```
[1] 3 5
```

```
b[-1]
```

```
[1] 3 5
```

Now, it is important to note that we have not been changing vector `b`. If we display the output of `b`, we can see that it still contains the 3 elements.

```
b
```

```
[1] 1 3 5
```

To change vector `b` we need to define `b` as vector `b` with the first element removed

```
b <- b[-1]
b
```

```
[1] 3 5
```

Vector `b` no longer contains 3 elements. Now, let's say we want to add an element to vector `b`.

```
c(5,b)
```

```
[1] 5 3 5
```

Here the `c()` function created a vector with the value 5 as the first element followed by the values in vector `b`

Or we can use the variable `a` that has a value of 5. Let's add this to vector `b`

```
b <- c(a,b)
b
```

```
[1] 5 3 5
```

What if you want to create a long vector with many elements? If there is a pattern to the sequence of elements in the vector then you can create the vector using `seq()`

```
seq(0, 1000, by = 4)
```

```
  [1]    0    4    8   12   16   20   24   28   32   36   40   44   48   52   56
 [16]   60   64   68   72   76   80   84   88   92   96  100  104  108  112  116
 [31]  120  124  128  132  136  140  144  148  152  156  160  164  168  172  176
 [46]  180  184  188  192  196  200  204  208  212  216  220  224  228  232  236
 [61]  240  244  248  252  256  260  264  268  272  276  280  284  288  292  296
 [76]  300  304  308  312  316  320  324  328  332  336  340  344  348  352  356
 [91]  360  364  368  372  376  380  384  388  392  396  400  404  408  412  416
[106]  420  424  428  432  436  440  444  448  452  456  460  464  468  472  476
[121]  480  484  488  492  496  500  504  508  512  516  520  524  528  532  536
[136]  540  544  548  552  556  560  564  568  572  576  580  584  588  592  596
[151]  600  604  608  612  616  620  624  628  632  636  640  644  648  652  656
[166]  660  664  668  672  676  680  684  688  692  696  700  704  708  712  716
[181]  720  724  728  732  736  740  744  748  752  756  760  764  768  772  776
[196]  780  784  788  792  796  800  804  808  812  816  820  824  828  832  836
[211]  840  844  848  852  856  860  864  868  872  876  880  884  888  892  896
[226]  900  904  908  912  916  920  924  928  932  936  940  944  948  952  956
[241]  960  964  968  972  976  980  984  988  992  996 1000
```

Vectors can only contain elements of the same "class".

```
d <- c(1, "2", 5, 9)
d
```

```
[1] "1" "2" "5" "9"
```

```
as.numeric(d)
```

```
[1] 1 2 5 9
```

1.6 Factors

Factors are special types of vectors that can represent categorical data. You can change a vector into a factor object using `factor()`

```
factor(c("male", "female", "male", "male", "female"))
```

```
[1] male   female male   male   female
Levels: female male
```

```r
factor(c("high", "low", "medium", "high", "low"))
```

```
[1] high   low    medium high   low
Levels: high low medium
```

```r
f <- factor(c("high", "low", "medium", "high", "low"),
            levels = c("high", "medium", "low"))
f
```

```
[1] high   low    medium high   low
Levels: high medium low
```

1.7 Lists

Lists are containers of objects. Unlike Vectors, Lists can hold different classes of objects.

```r
list(1, "2", 2, 4, 9, "hello")
```

```
[[1]]
[1] 1

[[2]]
[1] "2"

[[3]]
[1] 2

[[4]]
[1] 4

[[5]]
[1] 9

[[6]]
[1] "hello"
```

You might have noticed that there are not only single brackets, but double brackets `[[ ]]`

This is because Lists can hold not only single elements but can hold vectors, factors, lists, data frames, and pretty much any kind of object.

```r
l <- list(c(1,2,3,4), "2", "hello", c("a", "b", "c"))
l
```

```
[[1]]
[1] 1 2 3 4

[[2]]
[1] "2"

[[3]]
[1] "hello"

[[4]]
[1] "a" "b" "c"
```

You can see that the length of each element in a list does not have to be the same. To reference the elements in a list you need to use the double brackets `[[ ]]`.

```r
l[[1]]
```

```
[1] 1 2 3 4
```

To reference elements within list elements you use double brackets followed by a single bracket

```r
l[[4]][2]
```

```
[1] "b"
```

You can even give names to the list elements

```r
person <- list(name = "Jason",
               phone = "123-456-7890",
               age = 23,
               favorite_colors = c("blue", "red", "brown"))
person
```

```
$name
[1] "Jason"

$phone
[1] "123-456-7890"

$age
[1] 23

$favorite_colors
[1] "blue"  "red"   "brown"
```

And you can use the names to reference elements in a list

```
person[["name"]]
```

```
[1] "Jason"
```

```
person[["favorite_colors"]][3]
```

```
[1] "brown"
```

1.8 Data Frames

You are probably already familiar with data frames. SPSS and Excel uses this type of structure. It is just rows and columns of data. A data table! This is the format that is used to perform statistical analyses on.

So let's create a data frame so you can see what one looks like in RStudio

```
data <- data.frame(id = 1:10,
                   x = c("a", "b"),
                   y = seq(10,100, by = 10))
data
```

```
   id x   y
1   1 a  10
2   2 b  20
3   3 a  30
```

```
4   4 b  40
5   5 a  50
6   6 b  60
7   7 a  70
8   8 b  80
9   9 a  90
10 10 b 100
```

You can view the Data Frame by clicking on the object in the **Environment** window or by executing the command `View(data)`

Notice that it created three columns labeled `id`, `x`, and `y`. Also notice that since we only specified a vector of length 2 for `x` this column is coerced into 10 rows of repeating "a" and "b". All columns in a data frame must have the same number of rows.

You can use the `$` notation to reference just one of the columns in the data frame

```
data$y
```

```
 [1]  10  20  30  40  50  60  70  80  90 100
```

Alternatively you can use

```
data["y"]
```

```
     y
1   10
2   20
3   30
4   40
5   50
6   60
7   70
8   80
9   90
10 100
```

To reference only certain rows within a column

```
data$y[1:5]
```

```
[1] 10 20 30 40 50
```

```
data[1:5,"y"]
```

```
[1] 10 20 30 40 50
```

1.9 If...then Statements

If...then statements are useful for when you need to execute code only if a certain statement is TRUE. For instance,...

First we need to know how to perform logical operations in R

Figure 1.1: List of logical operators in R

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
!x	Not x
x | y	x OR y
x & y	x AND y
isTRUE(x)	test if X is TRUE

Okay, we have this variable `a`

```
a <- 5
```

Now let's say we want to determine if the value of `a` is greater than 3

```
a > 3
```

```
[1] TRUE
```

You can see that the output of this statement `a > 3` is `TRUE`

Now let's write an if…then statement. If `a` is greater than 3, then multiply `a` by 2.

```
if (a > 3) {
  a <- a*2
}
a
```

```
[1] 10
```

The expression that is being tested is contained in parentheses, right after the `if` statement. If this expression is evaluated as `TRUE` then it will perform the next line(s) of code.

The `{` is just a way of encasing multiple lines of code within one if statement. The lines of code then need to be closed of with `}`. In this case, since we only had one line of code `b <- a*2` we could have just written it as.

```
a <- 5
if (a > 3) a <- a*2
a
```

```
[1] 10
```

What if we want to do something to `a` if `a` is NOT greater than 3? In other words… if `a` is greater than 3, then multiple `a` by 2 else set `a` to `missing`

```
a <- 5
if (a > 3) {
  a <- a*2
} else {
  a <- NA
}
a
```

```
[1] 10
```

You can keep on chaining if…then… else… if… then statements together.

```r
a <- 5
if (is.na(a)) {
  print("Missing Value")
} else if (a < 0) {
  print("A is less than 0")
} else if (a > 3) {
  print("A is greater than 3")
}
```

```
[1] "A is greater than 3"
```

1.10 More Resources

For additional tips in the basics of coding R see:

https://r4ds.hadley.nz

https://ramnathv.github.io/pycon2014-r/visualize/README.html

https://www.datacamp.com/courses/free-introduction-to-r/?tap_a=5644-dce66f&tap_s=10907-287229

http://compcogscisydney.org/psyr/

References